

BMB2006 Veri Yapıları - Proje 4 Raporu

Blokzincir İşlem Ağlarının Analizi

1. Giriş ve Proje Amacı

Bu proje kapsamında, blokzincir sistemlerindeki karmaşık cüzdan hareketleri ve finansal işlem ağları sadeleştirilmiş bir model üzerinden incelenmiştir. Temel amacımız, kripto para transferlerini yönlü bir graf (Directed Graph) üzerinde modelleyerek; cüzdanlar arası fon akışını izlemek, Merkle Ağacı ile işlemlerin veri bütünlüğünü kriptografik olarak doğrulamak ve bu ağ üzerinde hedefe yönelik derinlemesine (DFS) ve sığ (BFS) analitik aramalar gerçekleştirmektir. Ayrıca proje, modern yazılım mühendisliği standartlarına uygun olarak tasarlanmış olup, büyük ölçekli verilerin test edilebilmesi amacıyla Python tabanlı bir Yapay Zeka (AI) mikroservisi ile entegre edilmiştir.

2. Sistem Mimarisi ve Eşzamanlılık (Microservices & Concurrency)

Proje gereksinimlerinde belirtilen "Eşzamanlılık ve Mikroservis Yaklaşımı" kriterini en üst düzeyde karşılamak amacıyla, Monolitik (tek parça) bir mimari yerine Dağıtık (Distributed) Mikroservis mimarisi tercih edilmiştir. Sistem üç ana bileşenden oluşmaktadır:

- **Çekirdek (Core) Backend (C# & .NET 9.0):** Graf veri yapılarının bellekte tutulduğu, arama algoritmalarının koştığı ve AIController ile WalletController köprülerinin barındırıldığı ana sunucudur. İşlemler senkron bloklanmaları önlemek amacıyla asenkron (async/await) olarak tasarlanmıştır.
- **AI Veri Simülasyon Servisi (Python & FastAPI):** Ana bellekten tamamen bağımsız ve izole çalışan bu servis, makine öğrenmesi algoritmaları (Pareto ve Lognormal dağılımlar) kullanarak gerçekçi cüzdan adresleri ve transfer verileri üretir.
- **İzole Ağ ve Konteynerizasyon (Docker):** Tüm servisler, bağımlılık çakışmalarını önlemek adına Dockerfile ile imajlanmış ve docker-compose kullanılarak bmb_network adında izole bir köprü ağ (bridge network) üzerinde tek bir komutla eşzamanlı çalışacak şekilde yapılandırılmıştır.

3. UML Sınıf Diyagramı ve Sistem Bileşenleri

Projenin genel mimarisi, Nesne Yönelimli Programlama (OOP) prensiplerine sadık kalınarak tasarlanmış olup, modülerliği artırmak amacıyla dört ana katmana (Package) ayrılmıştır:

A. Çekirdek Veri Yapıları (Core Data Structures - Backend)

- **WalletNode & TransactionEdge:** Graf yapısındaki düğümleri (cüzdanlar) ve yönlü kenarları (transferler) temsil eden temel nesnelerdir.
- **MyHashTable:** Cüzdan nesnelerini bellekte tutan ve $O(1)$ karmaşıklığında hızlı erişim sağlayan özel veri yapısıdır.
- **BlockchainGraph:** Sistemin ana yöneticisidir. İşlem listelerini, karma tablosunu barındırır ve BFS/DFS gibi arama algoritmalarını asenkron olarak çalıştırır.
- **MerkleTree:** İşlem listelerini alarak kriptografik özet (Hash) hesaplamaları yapar ve sistemin veri bütünlüğünü doğrular.

B. API Katmanı (Controllers)

- **WalletController:** Dış sistemlerin (Frontend) graf üzerindeki rota analizi (BFS/DFS) taleplerini karşılayan köprüdür.
- **AIController:** Sentetik veri üretimi için gerekli parametreleri alıp asenkron olarak yapay zeka servisini tetikler.

C. Harici Mikroservisler (External Microservices)

- **PythonAIService:** RESTful API üzerinden çalışan asenkron veri simülasyon motorudur. Gerçekçi ağ yapıları kurmak için Pareto ve Lognormal dağılım fonksiyonlarını barındırır.

D. Arayüz Katmanı (Frontend - Client)

- **CytoscapeGraphView:** Kullanıcı ile sistemin etkileşime girdiği tarayıcı tabanlı katmandır. C# API'sine gönderilen HTTP isteklerinin (AJAX) sonuçlarını alarak cüzdan bağlantılarını ve hedefe yönelik (Target) fon yollarını dinamik olarak haritalandırır.

A. Yönlü Graf (Directed Graph) Yapısı

Blokzincir ağındaki cüzdanlar ve transferler yönlü graf modeliyle simüle edilmiştir:

- **Düğüm (WalletNode):** Benzersiz cüzdan adresini (WalletAddress) ve bu cüzdandan yapılan transferleri tutan OutgoingTransactions listesini barındırır.
- **Kenarlar (TransactionEdge):** Cüzdanlar arası fon akışını temsil eder. TransactionId, hedef adres (ToAddress), transfer miktarı (Amount) ve zaman damgası (Timestamp) verilerini saklar.

Grafın bellekte tutulması için **Komşuluk Listesi (Adjacency List)** yöntemi seçilmiştir. Her düğüm, kendi üzerinden çıkan kenarları bir List<TransactionEdge> içinde dinamik olarak saklar.

B. Karma Tablo (MyHashTable) ve Veri Erişimi

Cüzdanların adres bilgisine göre hızlıca eklenmesi ve sorgulanabilmesi için 100 indeks kapasiteli özel bir karma tablo yapısı (public MyHashTable Wallets) kullanılmıştır. AddWallet metodunda cüzdan varlığı kontrol edilirken ve AddTransaction metodunda işlem eklenirken ilgili düğüme doğrudan Wallets[address] indeksi üzerinden erişilir.

```
public void AddWallet(string address)
{
    if (!Wallets.ContainsKey(address))
    {
        Wallets[address] = new WalletNode(address);
    }
}
```

- **Zaman Karmaşıklığı Analizi:**
 - **Ortalama Durum ($O(1)$):** Cüzdan adresleri tabloya homojen dağıldığında, Wallets[address] erişimi ve ContainsKey kontrolü sabit zamanda ($O(1)$) çalışır.
 - **En Kötü Durum ($O(N)$):** Tablo kapasitesinin üzerinde veri yüklendiğinde ve çakışmalar (collision) arttığında, aynı indekse düşen cüzdanlar ardışık aranacağı için karmaşıklık $O(N)$ olur.

C. Kriptografik Bütünlük Kontrolü (MerkleTree)

Bloktaki işlemlerin doğruluğunu teyit etmek amacıyla MerkleTree sınıfı yazılmıştır. Yaprak düğümlerden başlanarak işlemler ikili gruplar halinde SHA256 ile hash'lenir

ve tepedeki tek bir MerkleRoot değerine ulaşılır. Kodda tek kalan işlemler, Bitcoin mimarisine uygun olarak kendisiyle ikizlenerek işleme alınır:

```
while (currentLayer.Count > 1)
{
    List<string> nextLayer = new List<string>();
    for (int i = 0; i < currentLayer.Count; i += 2)
    {
        string left = currentLayer[i];
        string right = (i + 1 < currentLayer.Count) ? currentLayer[i + 1] :
left;
        nextLayer.Add(Hash(left + right));
    }
    currentLayer = nextLayer;
}
```

- **Zaman Karmaşıklığı Analizi:**

- **Ağaç İnşası (BuildTree):** Giriş olarak verilen N adet işlem için tüm katmanlar taranarak hash eşlemeleri yapıldığından $O(N)$ zaman karmaşıklığına sahiptir.
- **Doğrulama Karmaşıklığı:** İkili ağaç yapısı sayesinde, bir işlemin doğruluğunu kontrol etmek için sadece ilgili daldaki hash'lerin (Merkle Proof) incelenmesi yeterlidir. Bu işlem $O(\log N)$ adımda tamamlanır.

5. Arama Algoritmaları ve Analizi

BlockchainGraph sınıfı içinde, fon akışlarını takip etmek ve grafi analiz etmek için iki temel arama algoritması gerçekleştirilmiştir.

A. Enine Arama (BFS_TrackFundFlow)

BFS algoritması, bir cüzdandan çıkan paranın ağ üzerindeki yayılımını katman katman incelemek için yazılmıştır ve FIFO mantığına uygun olarak Queue<string> yapısını kullanır.

```
Queue<string> queue = new Queue<string>();
HashSet<string> visited = new HashSet<string>();

queue.Enqueue(startAddress);
visited.Add(startAddress);
```

- **Çalışma Mekanizması:** Sıradan çekilen her cüzdan rotam listesine eklenir. Ardından bu cüzdanın giden işlemleri (OutgoingTransactions) taranır. Daha önce ziyaret edilmeyen cüzdanlar (!visited.Contains), visited setine kaydedilerek kuyruğa eklenir.

- **Zaman Karmaşıklığı:** Graf üzerindeki tüm ulaşılabilir düğümler (V) kuyruğa girer ve bu düğümlerin bağlı olduğu kenarlar (E) birer kez incelenir. Bu nedenle zaman karmaşıklığı $O(V + E)$ 'dir.
- **Uzay Karmaşıklığı:** Kuyrukta bekleyen adresler ve visited kümesindeki benzersiz kayıtlar en kötü senaryoda tüm düğümleri kapsayabileceğinden $O(V)$ 'dir.

B. Derinlemesine Arama (DFS_DeepAnalysis)

DFS algoritması, fon akışının izole patikalar boyunca gidebileceği en uç noktaları ve hedef adresleri analiz etmek için yazılmıştır. LIFO mantığı doğrultusunda Stack<string> yapısını kullanır.

```
Stack<string> stack = new Stack<string>();
HashSet<string> visitedTransactions = new HashSet<string>();
```

- **Çalışma Mekanizması:** Blokzincir ağındaki çoklu transferleri ve döngüleri (cycle) doğru analiz edebilmek için standart düğüm ziyareti yerine **İşlem Bazlı Ziyaret (Transaction-Based Visited)** mantığı kurulmuştur. Yığıttan çekilen cüzdandan çıkan işlemler taranır. Eğer bir işlem (TransactionId) daha önce visitedTransactions setine eklenmemişse, işlem ziyaret edildi olarak işaretlenir ve hedeflenen cüzdan yığıta eklenir. Ayrıca sisteme entegre edilen minimum miktar (minAmount) filtresi, arama yolunu kesmeden sadece arayüze gidecek çıktıları filtreleyecek şekilde tasarlanmıştır.
- **Zaman Karmaşıklığı:** Çizgedeki tüm ulaşılabilir düğüm ve kenarları taradığı için zaman karmaşıklığı $O(V + E)$ 'dir.
- **Uzay Karmaşıklığı:** Ziyaret edilen işlemlerin kimlikleri hafızada tutulduğu için, uzay karmaşıklığı en kötü senaryoda graf üzerindeki işlem sayısına bağlı olarak $O(E)$ 'dir. Yığın yapısı ise en uzun transfer patikası kadar düğüm barındıracağından ek olarak $O(V)$ alan kullanır.

6. Yapay Zeka ve Sentetik Veri Modellemesi

A. Mikroservis Mimarisi ve Veri Doğrulama (FastAPI & Pydantic)

Blokzincir analiz platformunun grafik ve veri işleme yeteneklerini test etmek amacıyla, gerçek dünya ağ hareketlerini simüle eden bağımsız bir **Yapay Zeka (AI) ve Sentetik Veri Üretim Mikroservisi** geliştirilmiştir. Sistem mimarisinde C# Core API ile entegre çalışacak şekilde tasarlanan bu servis, asenkron ve hafif bir yapıda kurgulanmıştır.

- **İletişim Protokolü:** C# ana backend servisi, veri üretim taleplerini Python mikroservisine standart HTTP POST istek mekanizması üzerinden iletir. Servisler arası veri transferi tamamen asenkron olarak yürütülür.
- **FastAPI ve Veri Güvenliği:** Mikroservis omurgası olarak yüksek performanslı **FastAPI** çatısı tercih edilmiştir. Harici sistemlerden gelen verilerin doğrulanması ve tip güvenliğinin otomatik olarak sağlanması amacıyla Pydantic kütüphanesinden yararlanılmıştır.
- **Şema Yapısı (DataRequest):** BaseModel sınıfından türetilen DataRequest şeması, simülasyon için kritik iki parametreyi (wallet_count ve transaction_count) girdi olarak alır. Gelen ham JSON gövdesi, bu şemaya uymadığı takdirde çalışma zamanı hatalarını önlemek adına API katmanında otomatik olarak reddedilir.
- **Uç Noktalar (Endpoints):** Servis üzerinde iki adet API uç noktası konumlandırılmıştır. İlki servisin sağlık durumunu kontrol ederken, `@app.post("/api/ai/generate-synthetic-data")` dekoratörü ile tanımlanan ikinci uç nokta ise C# servisinden gelen parametrik istekleri karşılayarak veri üretim motorunu tetikler.

B. Sentetik Veri Üretim Algoritması ve Matematiksel Dağılımlar

Gerçekçi bir blokzincir ağı simüle edebilmek adına, rastgele veri üretimi yerine finansal piyasaların doğasına uygun matematiksel dağılım modelleri uygulanmıştır. Performans optimizasyonu amacıyla harici kütüphane bağımlılıkları minimize edilmiş, algoritmalar Python'ın yerleşik math ve random modülleriyle sıfırdan inşa edilmiştir.

1. Servet Eşitsizliği ve Balina Oluşumu (Pareto Dağılımı)

Sistem ilk olarak talep edilen miktarda benzersiz cüzdan adresi üretir. Ardından, gerçek kripto para piyasalarındaki sermaye yoğunlaşmasını (güç yasasını) taklit etmek amacıyla her cüzdana **Pareto Dağılımı** (80/20 kuralı) kullanılarak istatistiksel bir ağırlık atanır. Bu sayede simülasyonda az sayıda cüzdanın piyasayı domine eden "balina" hesaplara dönüşmesi, büyük çoğunluğun ise düşük sermayeli hesapları temsil etmesi sağlanır.

2. İşlem Aktivitesi ve Hesap Seçimi

Cüzdanların işlem yapma sıklıkları, yine Pareto ağırlık indeksleri üzerinden belirlenir. Yüksek ağırlıklı (balina) veya kurumsal nitelikteki hesapların ağ üzerinde işlem gerçekleştirme olasılığı matematiksel olarak artırılır. Her bir transfer işlemi için bu olasılık matrisi kullanılarak sistemden iki farklı cüzdan (gönderici ve alıcı) dinamik olarak seçilir.

3. Transfer Miktarlarının Belirlenmesi (Log-Normal Dağılım)

Blokzincir ağlarındaki transfer hacimleri homojen değildir. İşlem miktarlarının gerçekçiliğini sağlamak amacıyla **Log-Normal Dağılım** modeli entegre edilmiştir. Bu model sayesinde, mikroservis tarafından üretilen işlemlerin büyük bir kısmı düşük ve

"ortalama" miktarlardan oluşurken, çok büyük hacimli fon transferleri ise istatistiksel olarak nadir gerçekleşecek şekilde kurgulanmıştır.

C. Blokzincir Standartlarına Uyum ve Veri Serileştirme

Üretilen sentetik verilerin C# Core API katmanında doğrudan graf veri yapısına (BlockchainGraph) işlenebilmesi ve Merkle Ağacı tarafından doğrulanabilmesi için blokzincir standartları veri seviyesinde korunmuştur:

- **Kriptografik Kimliklendirme (SHA-256):** Üretilen her transfer işlemine özel, çakışma riski barındırmayan 256-bitlik benzersiz bir işlem kimliği (tx_id) SHA-256 algoritmasıyla hesaplanır.
- **Zaman Damgası Varyasyonu (Unix Epoch):** Simülasyonun zaman akışında doğrusal bir gerçekçilik yakalamak adına, belirlenen Unix başlangıç zaman damgasından ardışık adımlarla 1 ila 60 saniye arasında rastgele değerler düşülür. Bu sayede tüm işlemlerin aynı anda gerçekleşmesi engellenerek ağ trafiğine zamansal derinlik kazandırılır.
- **Veri Taslağı ve Çıktı Şablonu:** Elde edilen tüm metrikler, C# backend servisinin veri sözleşmesine tam uyumlu bir JSON şablonuna serileştirilir. Üretilen anlamlı sentetik veri kümesi; status, message, generated_wallets ve generated_transactions üst anahtarları (root keys) aracılığıyla ana sisteme kayıpsız bir şekilde geri döndürülür.

Sonuç olarak geliştirilen Python mikroservis bileşeni, karmaşık blokzincir ağ mekanizmalarını matematiksel doğrulukla simüle ederek veri yapıları testleri için optimize edilmiş, asenkron ve sürdürülebilir bir altyapı sağlamıştır.

7. Sonuç ve Değerlendirme

Bu proje kapsamında, blokzincir tabanlı finansal işlem ağlarının karmaşık yapısı; modern yazılım mimarisi, optimize edilmiş çekirdek veri yapıları ve yapay zeka destekli simülasyon modelleri kullanılarak başarıyla çözümlenmiştir.

Sistemin genel performansı ve hedeflere ulaşma durumu değerlendirildiğinde şu temel sonuçlar elde edilmiştir:

- **Mimari Başarı ve Eşzamanlılık:** Geleneksel monolitik tasarımlar yerine benimsenen Docker tabanlı, asenkron mikroservis mimarisi (C# .NET Core ve Python FastAPI), projenin ölçeklenebilirliğini artırmış ve diller arası (cross-language) veri iletişiminin sorunsuz çalışabileceğini kanıtlamıştır.
- **Veri Yapılarının Optimizasyonu:** Milyonlarca işlemin gerçekleşebileceği blokzincir ağları göz önüne alınarak tasarlanan MyHashTable yapısı, cüzdan sorgulamalarındaki arama maliyetini $O(1)$ seviyesine indirerek sistemin darboğaz (bottleneck) yaşamasını engellemiştir.

- **Algoritmik Doğruluk ve Güvenlik:** Ağ üzerindeki fon akışları, BFS ve DFS algoritmalarına entegre edilen "işlem bazlı (transaction-based) ziyaret" mantığı sayesinde döngülere (cycle) takılmadan eksiksiz bir şekilde haritalandırılmıştır. Aynı zamanda, Merkle Ağacı entegrasyonu ile bu transferlerin kriptografik bütünlüğü başarıyla güvence altına alınmıştır.
- **Gerçekçi Simülasyon (Sentetik Veri):** Yapay zeka servisinde uygulanan Pareto (80/20) ve Log-Normal dağılım modelleri, gerçek dünya blokzincir ağlarındaki borsaları ve "balina" (büyük sermayeli) cüzdan davranışlarını kusursuz bir şekilde simüle etmiştir. Bu sayede, yazılan graf algoritmalarının sıkı bağlantılı (strongly connected) ağ topolojilerinde nasıl davrandığı uçtan uca test edilebilmiştir.

Sonuç olarak; proje şartnamesinde belirtilen veri yapıları inşası, arama algoritmaları optimizasyonu ve arayüz görselleştirmesi hedefleri eksiksiz olarak yerine getirilmiştir. Veri yapıları ve dağıtık sistem mimarisi teorileri, sadece bir akademik proje düzeyinde kalmamış, endüstri standartlarına uygun, uçtan uca çalışan tam teşekküllü bir yazılım mühendisliği ürününe dönüştürülmüştür.